

Authentication and Authorization: Scenario-Based Interview Questions

Related files:

- [authentication-authorization-guide.md](#)
- [authentication-authorization-interview-notes.md](#)
- [authentication-authorization-top50-and-mcq.md](#)

This file focuses on scenario-based interview preparation. These are the kinds of questions where the interviewer wants to see how you think, how you choose an architecture, and whether you understand security tradeoffs.

How To Answer Scenario Questions

A strong interview answer usually includes:

1. Identify the type of application
2. Identify users and clients
3. Choose authentication model
4. Choose authorization model
5. Explain token/session strategy
6. Mention security protections
7. Mention tradeoffs

A good answer format:

First I would identify whether this is a browser app, SPA, mobile app, or service-to-service system.

Then I would choose the proper authentication flow.

After that I would define authorization using roles and permissions.

Finally I would explain token handling, session strategy, and security protections such as HTTPS, PKCE, password hashing, and least privilege.

Section 1: Banking App Auth Design

Scenario 1

You are designing authentication and authorization for a banking application with:

- customer login
- admin login
- account overview
- money transfer
- role-based access for support staff
- audit requirements

What architecture would you choose?

Strong answer

For a banking app, I would separate identity, API protection, and authorization rules clearly.

- Use a centralized authorization server or identity provider
- Use OpenID Connect for user authentication
- Use short-lived access tokens for APIs
- Use refresh tokens carefully with strong protection
- Use RBAC plus fine-grained permissions
- Use MFA for sensitive actions
- Use audit logging for critical operations
- Protect all traffic with HTTPS
- Hash passwords using a strong password encoder such as bcrypt

Example architecture

```
[Web/Mobile Client]
  |
  v
[Authorization Server / IdP]
  |
  v
[Banking APIs as Resource Servers]
  |
  v
[Database / Audit Logs]
```

Authentication design

- Customer login through OIDC
- Admin and support staff login through stronger policies
- MFA for admin and for high-risk customer actions

Authorization design

Use layered authorization:

- role examples:
 - CUSTOMER
 - SUPPORT
 - MANAGER
 - ADMIN
- permission examples:
 - account:read
 - transfer:create
 - transfer:approve
 - user:disable

Important banking protections to mention

- MFA for login and transfer approval
- session/device risk checks
- rate limiting
- transaction signing or step-up authentication for sensitive operations
- audit logs that cannot be easily altered
- account lockout or anomaly detection
- token expiration and revocation strategy

Good interview phrases

For a banking system I would not rely only on login success. I would add strong authorization checks, MFA, short-lived tokens, and detailed auditing for sensitive operations like fund transfer and admin actions.

Common mistakes

- using only role checks without action-level permissions
 - no MFA for sensitive actions
 - long-lived access tokens
 - no audit trail
 - storing passwords or secrets insecurely
-

Scenario 2

How would you design money transfer authorization in a banking app?

Strong answer

I would use authentication plus contextual authorization.

- User must be authenticated
- User must be authorized to operate on that account
- User must have enough balance
- Transfer may require OTP or MFA
- High-value transfers may need step-up authentication
- Every transfer should be logged with traceable audit information

Good interview phrasing

```
I would not treat money transfer as a simple role check. It needs account ownership validation, business rules, and often step-up verification such as OTP or MFA.
```

Scenario 3

How would you secure admin operations in a banking platform?

Strong answer

- separate admin login flow or stronger security policy
- mandatory MFA
- strict RBAC and least privilege
- fine-grained permissions for operations like user unlock, account freeze, or report access
- full audit logging
- possibly IP allowlisting or device trust policies

Section 2: Microservice Token Flow

Scenario 4

You have three microservices:

- User Service
- Order Service

- **Payment Service**

How would you secure communication between them?

Strong answer

I would distinguish between user-to-service and service-to-service communication.

- external user authentication should happen via an authorization server
- external clients get access tokens
- APIs validate tokens as resource servers
- internal service-to-service calls should use client credentials flow or another trusted machine identity mechanism

Example flow

```
User -> Client App -> Auth Server -> Access Token
Client App -> API Gateway / User Service
User Service -> Order Service (service token)
Order Service -> Payment Service (service token)
```

Why not reuse user tokens everywhere?

Sometimes propagating user identity is useful, but blindly passing the same external token through all internal services is not always ideal.

A stronger design may include:

- token exchange
- service-specific tokens
- least-privilege scopes per service

Important interview points

- validate issuer and audience
- define scopes per service
- use TLS between services
- avoid over-privileged tokens
- use short token lifetimes
- log inter-service requests with correlation IDs

Good interview phrasing

For microservices I would separate end-user access tokens from internal service identity. For service-to-service calls I would usually prefer client credentials or token exchange so each service gets only the permissions it actually needs.

Scenario 5

Should each microservice validate JWTs on its own, or should only the gateway do it?

Strong answer

The gateway can do early checks, but each resource server should still be able to enforce its own authorization.

Why:

- defense in depth
- direct service access may still happen
- service-specific authorization rules belong to the service

Balanced answer

- gateway can validate token format, issuer, expiration, and some common rules
- downstream services should still validate trust and enforce their own access rules

Bad answer to avoid

Only the gateway should validate tokens and internal services should trust everything blindly.

Scenario 6

What scopes would you define for microservices?

Strong answer

I would define scopes based on business capability and least privilege.

Examples:

- `user.read`
- `user.write`

- `order.read`
- `order.create`
- `payment.charge`
- `payment.refund`

If needed, I would also add internal service scopes for machine communication.

Section 3: SPA + Backend + PKCE Architecture

Scenario 7

You are building an Angular or React SPA with a Spring Boot backend API. How would you design authentication?

Strong answer

For a SPA, I would use Authorization Code Flow with PKCE.

- SPA redirects the user to the authorization server
- user authenticates there
- authorization code is returned
- SPA completes token acquisition with PKCE protection
- SPA uses access token to call Spring Boot API
- Spring Boot API acts as a resource server

High-level architecture

```
[SPA] <-> [Authorization Server]
[SPA] -> [Spring Boot API Resource Server]
```

Why PKCE?

- SPA is a public client
- it cannot safely store a client secret
- PKCE protects the authorization code flow

Important storage discussion

This is a common interview area.

Good answer:

- avoid unsafe long-term storage of sensitive tokens in browser storage if possible
- prefer secure architecture patterns

- understand CSRF/XSS tradeoffs
- if backend-for-frontend is used, server-managed session may reduce browser token exposure

Good interview phrasing

For a SPA I would use authorization code with PKCE because the SPA is a public client. The Spring Boot backend would be a resource server validating access tokens issued by the authorization server.

Scenario 8

Why is implicit flow generally avoided in modern SPA design?

Strong answer

Implicit flow is largely avoided because authorization code with PKCE provides better security properties and clearer token handling for modern public clients.

Good short answer

Modern best practice for SPA login is authorization code with PKCE, not implicit flow.

Scenario 9

Would you use only SPA token handling, or would you introduce a backend-for-frontend?

Strong answer

It depends on the risk profile and architecture.

- pure SPA approach may be acceptable in some systems
- for higher-security systems, a backend-for-frontend can reduce token exposure in the browser and simplify secure session handling

This is a good place to show balanced thinking instead of giving a one-size-fits-all answer.

Good interview phrasing

If the application has higher security requirements, I would consider a backend-for-frontend pattern so the browser handles less security-sensitive token logic directly.

Scenario 10

How does Spring Boot fit into a SPA + PKCE system?

Strong answer

Spring Boot usually acts as:

- a resource server if it protects APIs with bearer tokens
- sometimes a backend-for-frontend if the architecture uses server-managed sessions

If it is a resource server, I would configure:

- JWT validation
- issuer validation
- audience checks if required
- scope or authority-based endpoint protection

Example interview mention:

```
In Spring Security I would configure the backend with  
oauth2ResourceServer().jwt() and then map claims or scopes into authorities for  
endpoint authorization.
```

Section 4: Spring Security Config Troubleshooting

Scenario 11

You configured Spring Security, but every endpoint returns `401 Unauthorized`. What do you check first?

Strong answer

I would check:

1. whether the request actually includes credentials or bearer token
2. whether the token is expired
3. whether issuer URI or JWK configuration is correct
4. whether the filter chain matches the request
5. whether authentication is failing before authorization is even reached

Good phrasing

401 usually means authentication failed or was missing, so I would first inspect the incoming credentials, token validity, and resource server configuration.

Scenario 12

The user is authenticated, but gets `403 Forbidden`. What does that usually mean?

Strong answer

It usually means authentication succeeded, but authorization failed.

Things to check:

- missing role or authority
- wrong scope
- role prefix mismatch such as `ADMIN` vs `ROLE_ADMIN`
- method security rule mismatch
- endpoint matcher rules

Good short answer

401 usually means not authenticated, while 403 usually means authenticated but not authorized.

Scenario 13

Your Spring app uses `hasRole("ADMIN")`, but the user still cannot access the endpoint. Why might that happen?

Strong answer

In Spring Security, `hasRole("ADMIN")` usually checks for authority `ROLE_ADMIN`.

Possible issues:

- user only has `ADMIN` but not `ROLE_ADMIN`
- authorities are not loaded correctly
- custom JWT claim mapping is wrong
- method security and request security are checking different things

Good phrasing

I would verify how roles are mapped into `GrantedAuthority` objects because Spring usually expects the `ROLE_` prefix when using `hasRole`.

Scenario 14

JWT authentication is configured, but Spring does not recognize scopes as authorities. How do you reason about it?

Strong answer

I would check claim mapping.

- what claim contains the scopes
- how Spring converts those claims into authorities
- whether custom converter configuration is required

Interview point:

- authentication can succeed while authorization still fails if claims are not mapped into the expected authorities

Scenario 15

After enabling Spring Security, your public endpoints are no longer public. What could be wrong?

Strong answer

Possible causes:

- request matchers are ordered incorrectly
- `anyRequest().authenticated()` is catching everything before public matchers
- wrong path pattern
- multiple filter chains are interacting unexpectedly

Example reasoning

I would inspect the matcher order because a broad authenticated rule can override the intended public endpoint configuration if placed incorrectly.

Scenario 16

Your frontend calls the backend successfully from Postman, but browser requests fail. What security issue might be involved?

Strong answer

Possible causes:

- CORS misconfiguration
- CSRF issues for session-based flows
- missing browser credentials/cookies
- token not being attached correctly by frontend

Good interview phrasing

If Postman works but browser requests fail, I would investigate browser-specific concerns like CORS, CSRF, cookie settings, and frontend token handling.

Scenario 17

A login endpoint works, but protected APIs still fail with JWT validation errors. What do you check?

Strong answer

- wrong issuer URI
- incorrect JWK endpoint
- wrong audience
- expired token
- token from one environment being used against another environment
- clock skew issues

Scenario 18

Why might method security succeed differently from request-level security?

Strong answer

Because they are separate authorization layers.

- request-level rules protect URLs
- method-level rules protect method execution

If one layer is configured differently from the other, behavior may differ.

Good phrasing

```
I treat request-level and method-level security as complementary layers, so I verify both if authorization behavior seems inconsistent.
```

Section 5: Mixed Scenario Questions

Scenario 19

A company wants:

- employee SSO
- internal admin portal
- public mobile app
- partner API access

How would you think about the security architecture?

Strong answer

I would separate use cases by client type:

- employee portal: OIDC SSO
- admin portal: stronger policies and MFA
- mobile app: authorization code with PKCE
- partner API: OAuth2 client credentials or partner-specific client onboarding

I would use a centralized authorization server and protect each API as a resource server.

Scenario 20

A startup says, "We will just use one admin role for everything." What is the risk?

Strong answer

That creates over-privileged access and weak separation of duties.

Better approach:

- use RBAC
- break down permissions
- apply least privilege
- audit sensitive actions

Scenario 21

How would you answer if an interviewer asks, "When would you choose sessions over JWT?"

Strong answer

I would choose sessions for traditional server-rendered browser apps where server-managed login state is simpler and more natural.

I would choose JWT or bearer token-based resource server patterns for stateless APIs, mobile clients, SPAs, and distributed systems.

Balanced point:

- neither is automatically better in every case
 - the right choice depends on client type and architecture
-

Section 6: Compact Mock Interview Prompts

Use these as practice prompts.

Prompt 1

Design authentication for a banking app with web, mobile, and admin users.

Expected themes:

- centralized auth server
- OIDC
- MFA
- RBAC + permissions
- audit
- step-up auth

Prompt 2

Design secure service-to-service auth for five microservices.

Expected themes:

- client credentials
- token scopes
- least privilege
- TLS
- per-service validation

Prompt 3

Design login for a React SPA and Spring Boot API.

Expected themes:

- authorization code + PKCE
- SPA as public client
- Spring Boot resource server
- token validation

Prompt 4

Your Spring app returns 401 for every endpoint after enabling JWT.

Expected themes:

- missing token
- issuer URI
- JWK
- filter chain
- token expiration

Prompt 5

Your Spring app returns 403 for admin endpoints even though login works.

Expected themes:

- role vs authority
- `ROLE_` prefix
- matcher rules
- method security

Section 7: Fast Answer Templates

These are useful when you want to sound structured in an interview.

Template 1: Architecture answer

```
I would first identify the type of client and whether this is browser-based, mobile, SPA, or service-to-service. Then I would choose the authentication flow, define authorization using roles and permissions, and finally explain token handling, transport security, and auditing.
```

Template 2: Banking answer

For banking I would use strong authentication, MFA, fine-grained authorization, short-lived tokens, audit logging, and step-up verification for sensitive actions like transfers.

Template 3: Microservice answer

For microservices I would separate user authentication from service identity, use resource servers for API protection, and use client credentials or token exchange for internal calls with least-privilege scopes.

Template 4: SPA answer

For a SPA I would use authorization code with PKCE because the SPA is a public client and cannot securely keep a client secret. The backend API would be configured as a resource server.

Template 5: Troubleshooting answer

If I see 401, I first suspect authentication issues. If I see 403, I first suspect authorization issues. Then I inspect token validity, claims, role mapping, and Spring matcher or method-security configuration.

Section 8: Common Mistakes To Avoid In Scenario Interviews

- giving only definitions without architecture reasoning
 - saying JWT is always better than sessions
 - forgetting HTTPS
 - forgetting MFA in high-risk systems
 - using only roles without permissions
 - forgetting PKCE for public clients
 - confusing 401 and 403
 - not mentioning audit logs for financial/admin systems
 - assuming gateway-only validation is enough for all microservices
-

Official References

- Spring Security reference: <https://docs.spring.io/spring-security/reference/index.html>
- Spring Authorization Server reference: <https://docs.spring.io/spring-authorization-server/reference/index.html>
- OAuth 2.0 RFC 6749: <https://www.rfc-editor.org/rfc/rfc6749>
- Bearer Token RFC 6750: <https://www.rfc-editor.org/rfc/rfc6750>
- PKCE RFC 7636: <https://www.rfc-editor.org/rfc/rfc7636>
- OAuth 2.0 for Native Apps RFC 8252: <https://www.rfc-editor.org/rfc/rfc8252>
- OpenID Connect Core 1.0: https://openid.net/specs/openid-connect-core-1_0.html
- NIST RBAC project: <https://csrc.nist.gov/projects/role-based-access-control>